Durham University

# Version Control: *Command Line* Git

Chris Marcotte[1]
*Durham University*

[1]For errata and questions: christopher.marcotte@durham.ac.uk

# Overview



## Outline

*Git* -ting Started ..... 3
Command line *Git* ..... 5
Branching & Merging ..... 12
Conflicts ..... 20
Summary ..... 24

# Overview



## Outline

*Git* -ting Started ..... 3
Command line *Git* ..... 5
Branching & Merging ..... 12
Conflicts ..... 20
Summary ..... 24

## Learning Goals

- Ability to create a local repository with *Git*
- Ability to work in a repository
- Understanding of branching strategies
- Understanding of best practices for using *Git*

# Git *-ting Started*

# Installing *Git* locally



To use *Git* on the command line, you will need:

- A (`bash`) shell (and `ssh` for remote work)
- A *Git* installation
- A basic text editor, like `nano`

---

[1] Use `sudo apt install git nano` on Linux, or use `brew install git nano` on macOS after installing Homebrew.

[2] On recent Windows: Windows Terminal; on macOS: Terminal; on Linux: *you already know*.

# Installing *Git* locally



To use *Git* on the command line, you will need:

- A (`bash`) shell (and `ssh` for remote work)
- A *Git* installation
- A basic text editor, like `nano`

On Linux and macOS, these should be pre-installed.[1]

On Windows, you should use the Windows Terminal.[2]

---

[1] Use `sudo apt install git nano` on Linux, or use `brew install git nano` on macOS after installing Homebrew.

[2] On recent Windows: Windows Terminal; on macOS: Terminal; on Linux: *you already know*.

# Installing *Git* locally



To use *Git* on the command line, you will need:

- A (`bash`) shell (and `ssh` for remote work)
- A *Git* installation
- A basic text editor, like `nano`

On Linux and macOS, these should be pre-installed.[1]
On Windows, you should use the Windows Terminal.[2]

In a terminal run: `git --version`

---

[1] Use `sudo apt install git nano` on Linux, or use `brew install git nano` on macOS after installing Homebrew.

[2] On recent Windows: Windows Terminal; on macOS: Terminal; on Linux: *you already know*.

# *Command line* Git

# Essential *Git* commands



In this lecture I will discuss some essential *Git* commands:

# Essential *Git* commands



In this lecture I will discuss some essential *Git* commands:

- `git init`
- `git add`
- `git commit`
- `git branch`
- `git checkout`
- `git merge`

# Essential *Git* commands



In this lecture I will discuss some essential *Git* commands:

- `git init`
- `git add`
- `git commit`
- `git branch`
- `git checkout`
- `git merge`

These are the basic commands for using *Git* productively by **oneself**.

We will look at commands requiring GitHub in the next two lectures.

# Initializing a repository



In your terminal, move to your development directory[1]:

```
mkdir -p ./COMP51915_test
cd COMP51915_test/
git init .
```

1. Creates a new directory, `./COMP51915_test/`
2. Moves to the new directory, and
3. Initializes a new Git repository in the new directory.

---

[1] Do you have a development directory? For me, it is `~/Development/`

[2] Historically, the most fundamental branch was called *master*; in recent efforts to decolonize the field, people have moved to calling this branch *main*.

[3] The `.*` means 'all the hidden stuff in the current directory'.

# Initializing a repository



In your terminal, move to your development directory[1]:

```
mkdir -p ./COMP51915_test
cd COMP51915_test/
git init .
```

1. Creates a new directory, `./COMP51915_test/`
2. Moves to the new directory, and
3. Initializes a new Git repository in the new directory.

You may get a *hint* here about the branch name used[2], which you can change.

---

[1] Do you have a development directory? For me, it is `~/Development/`

[2] Historically, the most fundamental branch was called *master*; in recent efforts to decolonize the field, people have moved to calling this branch *main*.

[3] The `.*` means 'all the hidden stuff in the current directory'.

# Initializing a repository



In your terminal, move to your development directory[1]:

```
mkdir -p ./COMP51915_test
cd COMP51915_test/
git init .
```

1. Creates a new directory, `./COMP51915_test/`
2. Moves to the new directory, and
3. Initializes a new Git repository in the new directory.

You may get a *hint* here about the branch name used[2], which you can change.

Calling `ls .*` yields[3]:

```
.git:
  HEAD        description info        refs
  config      hooks       objects
```

---

[1] Do you have a development directory? For me, it is `~/Development/`

[2] Historically, the most fundamental branch was called *master*; in recent efforts to decolonize the field, people have moved to calling this branch *main*.

[3] The `.*` means 'all the hidden stuff in the current directory'.

# Adding files to a repository – *staging*



We now have a git repository, in our current directory.

Create two files, e.g. with `touch file1.c file2.py`.

How do we get *Git* to track these files?

[1] Except those under `gitignore`; this is useful if you're programming something in a single language and want everything included every time.

# Adding files to a repository – *staging*



We now have a git repository, in our current directory.

Create two files, e.g. with `touch file1.c file2.py`.

How do we get *Git* to track these files?

At the command line:

```
git add file1.c file2.py
```

Alternatively: `git add *` to add *all* files in the current directory to *Git*.[1]

[1] Except those under `gitignore`; this is useful if you're programming something in a single language and want everything included every time.

# `git add` limitations and caveats



The `git add` command updates the *Git* index with the current content.

# `git add` limitations and caveats



The `git add` command updates the *Git* index with the current content.

You can check that a file has been added correctly with `git status`:

```
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
   new file:   file1.c
   new file:   file2.py
```

This is telling us that:

- we are on the *main* branch
- there are no commits
- we've staged the files to be committed

# `git add` limitations and caveats



The `git add` command updates the *Git* index with the current content.

You can check that a file has been added correctly with `git status`:

```
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
   new file:   file1.c
   new file:   file2.py
```

This is telling us that:

- we are on the *main* branch
- there are no commits
- we've staged the files to be committed

*Git* has four file status'; these files went from *untracked* to *modified* to *staged*.

# Committing to a repository



Our files have been staged with `git add`, but we have not yet *committed* them.[1]

[1] On my terminal, before the commit the directory was adorned with a ✗, afterward with a ✓.

# Committing to a repository



Our files have been staged with `git add`, but we have not yet *committed* them.[1]

To commit the modified files to the repository,

```
git commit -m "adding file1.c file2.py to repo."
```

The *commit message* is a **brief** description of the changes made in the commit.

> Running `git log` lists all commits made to a repository in reverse order.

[1] On my terminal, before the commit the directory was adorned with a ✗, afterward with a ✓.

# *Interlude*: Good commits



There is, in effect, a style guide for commits endorsed by *Git*.

1. Commit small changes consistently, rather than large changes occassionally
2. Make your changes address one particular issue, rather than several
3. ensure your commit messages are descriptive, on-topic, and conform to the expectations of the project

Running `git diff --check` before committing, then *Git* will notify you of some common issues in your files before you commit them.

# Branching & Merging

# Branches

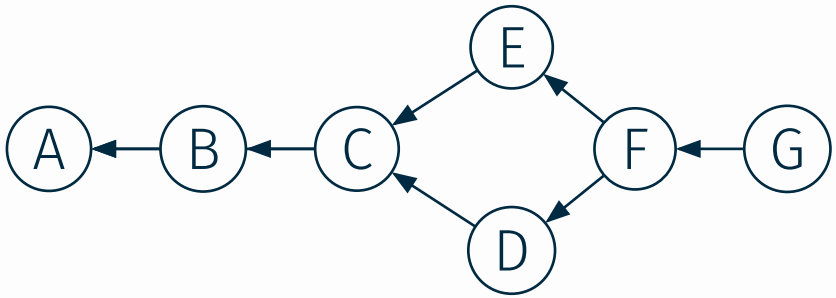


**Branches** are divergent development histories of a *Git* managed repository.

In *Git*, the branching model is very lightweight so things can be done quickly.

When modifying a repository, you should:

1. *branch*,
2. *edit*, and finally
3. *merge*.




*Branching* and *merging* are critical tools for maintaining a repository, as they ensure your working directory is clean and separate from `main`.

# Creating Branches



A new branch `newbranch` can be made as easily as `git branch newbranch`.

But this only *creates* the new branch – switch using `git checkout newbranch`.

# Creating Branches



A new branch `newbranch` can be made as easily as `git branch newbranch`.

But this only *creates* the new branch – switch using `git checkout newbranch`.

From here, we can:

1. edit a file (e.g. with `echo "import numpy as np" >> file2.py`), and
2. add a file (e.g. `touch file3.h`), and
3. stage the changes (with `git add file2.py file3.h`), and even
4. commit changes to `newbranch` (e.g. `git commit -m "commit-ing crimes"`)

# Branch Management



Return to the main branch with `git checkout main` and look with `cat file2.py`.

We'll find that `file2.py` is *empty* on `main`, and `file3.h` is *missing* entirely.

[1] Indeed, *Git* will not let you switch to a different branch if your directory is modified and unstaged.

# Branch Management



Return to the main branch with `git checkout main` and look with `cat file2.py`.

We'll find that `file2.py` is *empty* on `main`, and `file3.h` is *missing* entirely.

Changing branches *modifies* the content in the working directory.[1]

To get the changes we committed in `newbranch` to be reflected in the `main` branch we need to discuss *merging* and the `git merge` command.

[1] Indeed, *Git* will not let you switch to a different branch if your directory is modified and unstaged.

# Merging



The command to merge content from `newbranch` into the current branch is:

```
git merge newbranch
```

# Merging



The command to merge content from `newbranch` into the current branch is:

```
git merge newbranch
```

Because we have no conflicts, *Git* merges using *fast-forward*.

Fast-forward requires your current commit (`main`) occur in the history of your target commit (`newbranch`) – and works by moving the pointer for the current commit to the target commit – this is very cheap!

# Merging



The command to merge content from `newbranch` into the current branch is:

```
git merge newbranch
```

Because we have no conflicts, *Git* merges using *fast-forward*.

Fast-forward requires your current commit (`main`) occur in the history of your target commit (`newbranch`) – and works by moving the pointer for the current commit to the target commit – this is very cheap!

There are other methods *Git* will use to manage a more complex merge, but these are selected automatically – *usually*.

# Branching Strategies



There are a number of branching strategies used for *Git*.

We can not cover all possibilities; some of the more widely used are:

- long-running branches, discriminated by *code stability*
  - like **production**, **beta**, **stable**, **development**, etc.
- short-lived branches, discriminated by *feature implementation*
  - e.g. for the implementation of a **new feature**, fixing a bug, etc.

# Branching Strategies



There are a number of branching strategies used for *Git*.

We can not cover all possibilities; some of the more widely used are:

- long-running branches, discriminated by *code stability*
  - like **production**, **beta**, **stable**, **development**, etc.
- short-lived branches, discriminated by *feature implementation*
  - e.g. for the implementation of a **new feature**, fixing a bug, etc.

*Remote* repositories and collaboration give us even more options.

# Branching Strategies



There are a number of branching strategies used for *Git*.

We can not cover all possibilities; some of the more widely used are:

- long-running branches, discriminated by *code stability*
  - like **production**, **beta**, **stable**, **development**, etc.
- short-lived branches, discriminated by *feature implementation*
  - e.g. for the implementation of a **new feature**, fixing a bug, etc.

*Remote* repositories and collaboration give us even more options.

*Git* differs from most VC systems because of the inexpensive branching model – this makes multiple short-lived branches viable and uniquely *Git*.

# Protecting Branches



*Git* has a number of ways to protect branches from being modified.

The most common is to *protect* the `main` branch, so that it can only be modified by merging in changes from other branches.

# Protecting Branches



*Git* has a number of ways to protect branches from being modified.

The most common is to *protect* the `main` branch, so that it can only be modified by merging in changes from other branches.

This is done by setting up a *branch protection rule* on GitHub, which will prevent any changes to the `main` branch unless they are merged in from another branch.

# Protecting Branches



*Git* has a number of ways to protect branches from being modified.

The most common is to *protect* the `main` branch, so that it can only be modified by merging in changes from other branches.

This is done by setting up a *branch protection rule* on GitHub, which will prevent any changes to the `main` branch unless they are merged in from another branch.

This is a good way to ensure that your `main` branch is always stable and ready for production – e.g. for release to the public.

# Deleting Branches



We can delete a branch using `git branch -d newbranch`.

You should typically delete a branch after it's been merged into `main` – the label for both branches will point to the same commit from then on.

# Deleting Branches



We can delete a branch using `git branch -d newbranch`.

You should typically delete a branch after it's been merged into `main` – the label for both branches will point to the same commit from then on.

This is part of repository maintenance, though you might have good reason for keeping a branch around.

If you are wondering if you should do something as part of maintenance, ask yourself:

> *Will doing this change now help avoid confusion later?*

# Conflicts

# Merge Conflicts



Thus far we've seen simple merges. Merge conflicts occur between two branches have modified a file in incompatible ways. Let's create a merge conflict!

# Merge Conflicts



Thus far we've seen simple merges. Merge conflicts occur between two branches have modified a file in incompatible ways. Let's create a merge conflict!

1. Create a new branch `git branch newbranch` and
   - modify `file2.py`,
   - then `git add file2.py`,
   - and `git commit`.
2. On branch `main`,
   - edit `file2.py`,
   - then `git add file2.py`
   - and `git commit`.

# Example: Creating a Merge Conflict



When we try to `git merge newbranch` we'll get a failure notice:

```
Auto-merging file2.py
CONFLICT (content): Merge conflict in file2.py
Automatic merge failed; fix conflicts and then commit the result.
```

# Example: Creating a Merge Conflict



When we try to `git merge newbranch` we'll get a failure notice:

```
Auto-merging file2.py
CONFLICT (content): Merge conflict in file2.py
Automatic merge failed; fix conflicts and then commit the result.
```

The merge has been paused while you fix your repository.

Use `git status`:

```
Unmerged paths:
(use "git add <file>..." to mark resolution)
      both modified:   file2.py

no changes added to commit (use "git add" and/or "git commit -a")
```

# Resolving Merge Conflicts



*Git* has helpfully added lines to point out the conflict.

Use `cat file2.py`:

```
<<<<<<< HEAD
return np.sin(x)-x
=======
return x*x*x
>>>>>>> newbranch
```

You resolve the merge conflict by :

1. modifying the section of the file by deleting the appropriate lines,
2. staging the file with `git add` – this marks the conflict as resolved, and
3. finishing the commit with `git commit`.

# Summary

# *Git* Command Line Interface Summary



## Essential Commands

- `git init`
- `git add` / `git rm`
- `git commit`
- `git branch`
- `git checkout`
- `git merge`

## Good Practices

- Commit small changes consistently, rather than large changes occasionally
- Make your changes address one particular issue, rather than several
- Ensure your commit messages are descriptive, specific, and short

# *Git* Command Line Interface Summary



## Essential Commands

- `git init`
- `git add` / `git rm`
- `git commit`
- `git branch`
- `git checkout`
- `git merge`

## Good Practices

- Commit small changes consistently, rather than large changes occasionally
- Make your changes address one particular issue, rather than several
- Ensure your commit messages are descriptive, specific, and short

Mastering command line *Git* is essential for tracking your work in repositories, and is a skill you will use throughout the programme.